

```
message Role{
    // Role name
    string name = 1;

    // Name of corresponding policy
    string policy_name = 2;
}
```

Addressing

All identity data is stored under the special namespace “00001d”.

For each policy, the address is formed by concatenating the namespace, the special policy namespace of ‘00’, and the first 62 characters of the SHA-256 hash of the policy name:

```
>>> "00001d" + "00" + hashlib.sha256(policy_name.encode()).
hexdigest()[:62]
```

Address construction for roles follows a pattern similar to the address construction in the settings namespace. Role names are broken into four parts, where parts of the string are delimited by the “.” character. A short hash is computed for each part. For the first part, the first 14 characters of the SHA-256 hash are used. For the remaining parts, the first 16 characters of the SHA-256 hash are used. The address is formed by concatenating the identity namespace ‘00001d’, the role namespace ‘01’, and the four short hashes.

For example, the address for the role *client.query_state* is constructed as follows:

```
>>> "00001d" + "01" + hashlib.sha256('client'.encode()).
hexdigest()[:14] + \
    hashlib.sha256('query_state'.encode()).hexdigest()[:16] + \
    hashlib.sha256('').hexdigest()[:16] + \
    hashlib.sha256('').hexdigest()[:16]
```

Transaction payload

Identity transaction family payloads are defined by the following protocol buffer code:

```
message IdentityPayload {
    enum IdentityType {
        POLICY = 0;
        ROLE = 1;
    }

    // Which type of payload this is for
    IdentityType type = 1;

    // Serialize bytes of a role or a policy
    bytes data = 2;
}
```

Transaction header

Inputs and outputs

The inputs for identity family transactions must include:

- The address of the setting *sawtooth.identity.allowed_keys*
- The address of the role or policy being changed
- If setting a role, the address of the policy to assign to the role

The outputs for identity family transactions must include:

- The address of the role or policy being changed

Dependencies

None.

Family

- family_name: “sawtooth_identity”
- family_version: “1.0”

Execution

Initially, the transaction processor gets the current values of *sawtooth.identity.allowed_keys* from the state. The public key of the transaction signer is checked against the values in the list of allowed keys. If it is empty, no roles or policy can be updated. If the transaction signer is not among the allowed keys, the transaction is invalid. Whether this is a role or a policy transaction is checked by looking at the *IdentityType* in the payload.

If the transaction is for setting a policy, the data in the payload is parsed to form a policy object. This is then checked to make sure it has a name and at least one entry. If either is missing, the transaction is considered invalid. If the policy is determined to be whole, the address for the new policy is fetched. If there is no data found at the address, a new policy list object is created, the new policy is added, and the policy list is applied to state. If there is data, it is parsed into a policy list. The new policy is added to the policy list, replacing any policy with the same name, and the policy list is applied to state.

If the transaction is for setting a role, the data in the payload is parsed to form a role object, which is then checked to make sure it has a *name* and a *policy_name*. If either is missing, the transaction is considered invalid. The *policy_name* stored in the role must match a policy already stored in state; if no policy is found stored at the address created by the *policy_name*, the transaction is invalid. If the policy exists, the address for the new role is fetched. If there is no data found at the address, a new *RoleList* object is created, the new role is added, and the policy list is applied to state. If there is data, it is parsed into a *RoleList*. The new role is added to the role list, replacing any role with the same name, and the role list is applied to state. **END** 

By: Ashwini Kulkarni

The author is an independent consultant with experience in training and an active interest in emerging technologies like blockchain and containerisation. She's also certified in cyber law and cyber forensics.